

Απαλλακτική Εργασία – Real Time Strategy

Μάθημα: Γραφικά και Εικονική Πραγματικότητα
Διδάσκων Καθηγητής: Κωνσταντίνος Μουστάκας
Ονοματεπώνυμο: Κωνσταντίνος Κανελλόπουλος
Αριθμός Μητρώου: 1100881
Έτος: 2025 - 2026



Πίνακας Περιεχομένων

Εισαγωγή: Πρώτη Ημέρα

I: Γένεσις – Γη και Φως

- 1.1 Η Ανατολή ενός Κόσμου Κρυμμένου στην Ομίχλη (Terrain Architecture)
- 1.2 Έστω Φως (Lighting & Shadows)
- 1.3 Το Βλέμμα του Παντεπόπτη (RTS Camera)

II: Η Δημιουργία της Ζωής

- 2.1 Η Επικοινωνία με τον Δημιουργό (GUI)
- 2.2 Τα Κτίσματα και τα Units (Buildings & Units)
- 2.3 Η λογική των Units (State Machine Logic)
- 2.4 Ο Μόχθος της Επιβίωσης (Resource Gathering)
- 2.5 Η Τέχνη του Πολέμου (Combat Logic)
- 2.6 Το Φωτοστέφανο της Επιλογής (Selection Ring & Animation)

III: Έλεγχος του Βασιλείου

- 3.1 Ενσάρκωση (First/Third Person Camera)
- 3.2 Η Ανέγερση (Gradual Construction)
- 3.3 Η Αόρατη Χείρα (Ray Casting)
- 3.4 Η Μαζική Κλήση (Drag Box Selection)

IV: Οδός

- 4.1 Η Χαρτογράφηση (Grid System)
- 4.2 Ο Προφήτης του Δρόμου (A* Algorithm)
- 4.3 Η Θεία Πρόνοια (Dynamic Updates & Smoothing)

V: Οικονομία

- 5.1 Η Απόρριψη του Περιττού (Frustum Culling)
- 5.2 Το Πλήθος των Οντοτήτων (Instancing)

VI: Αποκάλυψις

- 6.1 Η Εντολή της Καταστροφής
- 6.2 Τα Σημάδια στη Γη (Terrain Deformation)

VII: Μυστήριο (Fog of War)

- 7.1 Το Άγνωστο (Unexplored/Explored/Visible)
- 7.2 Η Αποκάλυψη της Αλήθειας (Visibility Calculation)

VIII: Ο Σκοπός (Capture the Flag)

- 8.1 Η Ιερή Περιοχή
- 8.2 Η Κυριαρχία

IX: Η Δομή της Κτίσης (Ανάλυση Αρχιτεκτονικής)

- 9.1 Η Ιεραρχία των Αρχείων
- 9.2 Η Τάξη έναντι του Χάους (Refactoring & Code Structure)

X: Ευαγγελισμός – Η Διάδοση του Κόσμου (Cross-platform & Release)

- 10.1 Η Γέφυρα των Κόσμων (Cross-Platform Compatibility)
- 10.2 Η Παράδοση στους Πιστούς (Release Management)
- 10.3 Το Βιβλίο της Ιστορίας (Version Control)

Επίλογος: Η Έβδομη Ημέρα

Πρόσθετο υλικό:

Executables:

<https://drive.google.com/drive/folders/19fxybPmPEIM3Ejb7JAfAJBaCKvE66S0s?usp=sharing>

GithubRepo:

<https://github.com/BenizelosOEllhn/Real-Strategy-Game-Upatras>

Αν κατεβάσετε τον κώδικα και προσπαθήσετε να τον τρέξετε δεν θα δουλέψει

Εισαγωγή: Πρώτη Ημέρα

Στην αρχή, η οθόνη ήταν σκοτεινή. Δεν υπήρχε γη, ούτε φως, ούτε ζωή· παρά μόνο το κενό του `main()`. Η διαδικασία ανάπτυξης ενός παιχνιδιού στρατηγικής πραγματικού χρόνου RTS δεν είναι απλώς μια άσκηση προγραμματισμού είναι μια πράξη ψηφιακής κοσμογονίας. Ως δημιουργός, καλείσαι να ορίσεις τους φυσικούς νόμους, να σμιλεύσεις τα βουνά από το μηδέν, να χαράξεις την πορεία των ποταμών και, τελικά, να εμψύσεις «πνοή» σε άψυχα πολυγωνικά μοντέλα, χαρίζοντάς τους νοημοσύνη και σκοπό.

Η παρούσα εργασία περιγράφει το χρονικό αυτής της δημιουργίας. Από την procedural γένεση του εδάφους μέχρι την πολυπλοκότητα της τεχνητής νοημοσύνης (Pathfinding & FSM), και από την αρχιτεκτονική του Rendering μέχρι τη δικτυακή επικοινωνία. Στις σελίδες που ακολουθούν, παρουσιάζεται όχι μόνο το τελικό αποτέλεσμα, αλλά και η αρχιτεκτονική σκέψη πίσω από κάθε απόφαση: πώς το χάος των δεδομένων μετατράπηκε σε τάξη, πώς η στασιμότητα έγινε κίνηση και πώς ο παίκτης μετατρέπεται τελικά σε έναν παντεπόπτη στρατηγό.

1. Διασφάλιση Συμβατότητας και Διαδικασίας Ανάπτυξης

Η ανάπτυξη και υποστήριξη σε macOS ανέδειξε σημαντικές διαφοροποιήσεις στη διαχείριση βιβλιοθηκών, στη μεταγλώττιση και στη δομή των build συστημάτων. Κατά τη διάρκεια των εργαστηρίων, βρέθηκαν αρκετές φορές αντιμέτωπος με προβλήματα που σχετίζονταν με τις βιβλιοθήκες και το αρχείο `CMakeLists.txt`. Στην αρχή, οι δυσκολίες αυτές με καθυστερούσαν σημαντικά, καθώς κάθε ασυμβατότητα ή παρωχημένη ρύθμιση οδηγούσε σε σφάλματα μεταγλώττισης που δεν ήταν πάντα προφανή.

Ωστόσο, όσο προχωρούσε το εξάμηνο και αφιέρωνα χρόνο στο να κατανοήσω ουσιαστικά τον τρόπο λειτουργίας του CMake — πώς ορίζονται τα targets, πώς γίνεται το linking των βιβλιοθηκών και πώς διαχειρίζεται τις διαφορετικές πλατφόρμες — άρχισα να αντιμετωπίζω τα προβλήματα διαφορετικά. Αντί να προσπαθώ να διορθώσω αποσπασματικά ένα ήδη μπερδεμένο build σύστημα, επέλεγα συχνά να ξαναστήνω από την αρχή τα αρχεία ρύθμισης και τη σύνδεση των βιβλιοθηκών σε κάθε εργαστήριο.

Αυτή η διαδικασία, παρότι αρχικά χρονοβόρα, με βοήθησε να αποκτήσω ουσιαστική κατανόηση της δομής του έργου και να αισθάνομαι μεγαλύτερη αυτοπεποίθηση στον έλεγχο του περιβάλλοντος ανάπτυξης.

Η εμπειρία αυτή ανέδειξε την ανάγκη για ορισμένες βελτιώσεις συμβατότητας, οι οποίες θα μπορούσαν να ενσωματωθούν μελλοντικά σε επόμενο εργαστήριο, ώστε να αποφεύγονται αντίστοιχα προβλήματα. Σε περίπτωση που στο επόμενο εξάμηνο υπάρξουν φοιτητές που εργάζονται σε macOS αντί για Visual Studio 2019, θα ήταν ιδιαίτερη χαρά μου να συμβάλω στη δημιουργία ενός πιο ολοκληρωμένου και κατανοητού οδηγού (guide) εγκατάστασης και ρύθμισης του περιβάλλοντος ανάπτυξης.

Μερικές από τις διορθώσεις που υλοποιούσα:

Προστέθηκε φάκελος `.vscode` με κατάλληλες ρυθμίσεις για CMake Tools, IntelliSense και Debugging configuration. Οι ρυθμίσεις αυτές δεν ήταν απαραίτητες για τη διαδικασία build, ωστόσο βελτίωσαν σημαντικά την αξιοπιστία της ανάπτυξης και τη διαδικασία αποσφαλμάτωσης.

Αναβάθμιση και Εκσυγχρονισμός CMakeLists.txt

Το αρχικό αρχείο CMakeLists.txt απαιτούσε cmake_minimum_required(VERSION 3.0) και χρησιμοποιούσε παρωχημένες πολιτικές (CMP0026, CMP0042), οι οποίες δεν υποστηρίζονται σε σύγχρονες εκδόσεις CMake. Πραγματοποιήθηκαν οι εξής αλλαγές:

- Αναβάθμιση της απαίτησης CMake σε έκδοση ≥ 3.5 για συμβατότητα με σύγχρονες εκδόσεις.
- Κατάργηση χρήσης της πολιτικής CMP0042.
- Αποφυγή χρήσης της ιδιότητας LOCATION σε targets, καθώς δεν υποστηρίζεται πλέον.

Η παλιά βιβλιοθήκη rpanlik-cmake-modules χρησιμοποιούσε get_target_property(... LOCATION). Η συγκεκριμένη πρακτική αντικαταστάθηκε με γεννήτρια έκφραση (generator expression): set(USERFILE_COMMAND "\$<TARGET_FILE:\${_targetname}>") η οποία είναι σύμφωνη με τα σύγχρονα πρότυπα CMake.

Αναβάθμιση Βιβλιοθηκών

Η αρχική έκδοση GLFW 3.2.1 παρουσίαζε προβλήματα συμβατότητας με macOS και σύγχρονο Clang, κυρίως λόγω λανθασμένης αναγνώρισης αρχείων Objective-C (.m).

Για τον λόγο αυτό:

- Αντικαταστάθηκε με GLFW 3.4
- Τροποποιήθηκαν τα include_directories ώστε να δείχνουν στο νέο φάκελο
- Έγινε αναβάθμιση της GLM σε έκδοση $\geq 0.9.9$

Στο προηγούμενο εργαστήριο είχε επιλεγεί χειροκίνητη σύνδεση βιβλιοθηκών μέσω Homebrew. Ωστόσο, αυτή η προσέγγιση δημιουργούσε προβλήματα φορητότητας. Στην παρούσα υλοποίηση, επιλέχθηκε η χρήση βιβλιοθηκών που περιλαμβάνονται απευθείας στο project, εξασφαλίζοντας μεγαλύτερη σταθερότητα και ανεξαρτησία από το εκάστοτε σύστημα.

Αντιμετώπιση Σφάλματος Shader

Παρουσιαζόταν το σφάλμα: Can't open shader file: transformation.vertexshader

Το πρόβλημα οφειλόταν στο γεγονός ότι τα αρχεία shader δεν αντιγράφονταν στον φάκελο εκτέλεσης (build directory). Η λύση περιλάμβανε την προσθήκη add_custom_command στο CMake, το οποίο:

- Αντιγράφει όλα τα .vertexshader και .fragmentshader
- Τα τοποθετεί στον φάκελο build μετά τη δημιουργία του εκτελέσιμου

Με τον τρόπο αυτό, διασφαλίστηκε ότι τα shaders είναι πάντοτε διαθέσιμα κατά την εκτέλεση.

Κεφάλαιο 1: Η Γένεση του Κόσμου

«Εκ του Χάους, το Φως και η Γη»

1.1 Terrain Architecture - Η Ανατολή ενός Κόσμου Κρυμμένου στην Ομίγλη

Η δημιουργία του εδάφους αποτέλεσε την πρώτη και πιο χρονοβόρα πρόκληση αυτού του ταξιδιού. Αντί να χρησιμοποιήσω ένα έτοιμο 3D μοντέλο, επέλεξα να προσπαθήσω μέσω Procedural Generation μέσω κώδικα C++. Αυτή η προσέγγιση μου επέτρεψε να έχω απόλυτο έλεγχο σε κάθε vertex και να διαμορφώσω τη συμμετρία που απαιτούσε το gameplay, διατηρώντας όμως τη φυσικότητα του τοπίου.

1. Η Λογική της Υψομετρικής Συνάρτησης (getHeight)

Ο πυρήνας του εδάφους δεν είναι ένα στατικό mesh, αλλά μια μαθηματική συνάρτηση $getHeight(x, z)$ που υπολογίζει το ύψος y για κάθε σημείο του χάρτη.

- Φυσικά Σύνορα (Ακρωτήριο): Για να αποφύγω τους "αόρατους τοίχους" στα όρια του χάρτη, σχεδίασα το έδαφος ώστε να καταλήγει φυσικά στη θάλασσα. Δημιούργησα ένα Ακρωτήριο (που στενεύει σταδιακά καθώς προχωράμε προς τον βορρά (από το $zBase$ στο $zTip$), χρησιμοποιώντας μια συνάρτηση εξομάλυνσης (clamping & smoothing) για να σβήνει ομαλά στον ωκεανό.
- Οροσειρές και Πεδιάδες: Χρησιμοποίησα συνδυασμούς ημιτονοειδών κυμάτων ($\sin$, $\cos$) για να δώσω τυχαιότητα και "θόρυβο" στο έδαφος, ώστε να μη φαίνεται επίπεδο και τεχνητό.

2. Το Υδάτινο Σύστημα: Από τις Πηγές στον Ωκεανό

Η πιο σύνθετη γεωμετρική δομή ήταν η διαδρομή του νερού. Η αρχιτεκτονική βασίστηκε στα εξής στάδια:

1. Η Λίμνη (Πηγή): Στο κέντρο του βουνού, δημιούργησα μια λίμνη με ακανόνιστη ακτίνα βασισμένη σε τριγωνομετρικές συναρτήσεις, ώστε να μοιάζει φυσική.
2. Τα Ποτάμια (Ροή): Από τη λίμνη ξεκινούν δύο ποτάμια που διασχίζουν τον χάρτη συμμετρικά. Μέσω της συνάρτησης $carveRiver$, "σκάβω" το έδαφος αφαιρώντας ύψος κατά μήκος μιας ημιτονοειδούς διαδρομής, καταλήγοντας τελικά στη θάλασσα.

3. Το Τεχνικό Πρόβλημα του "Water Plane"

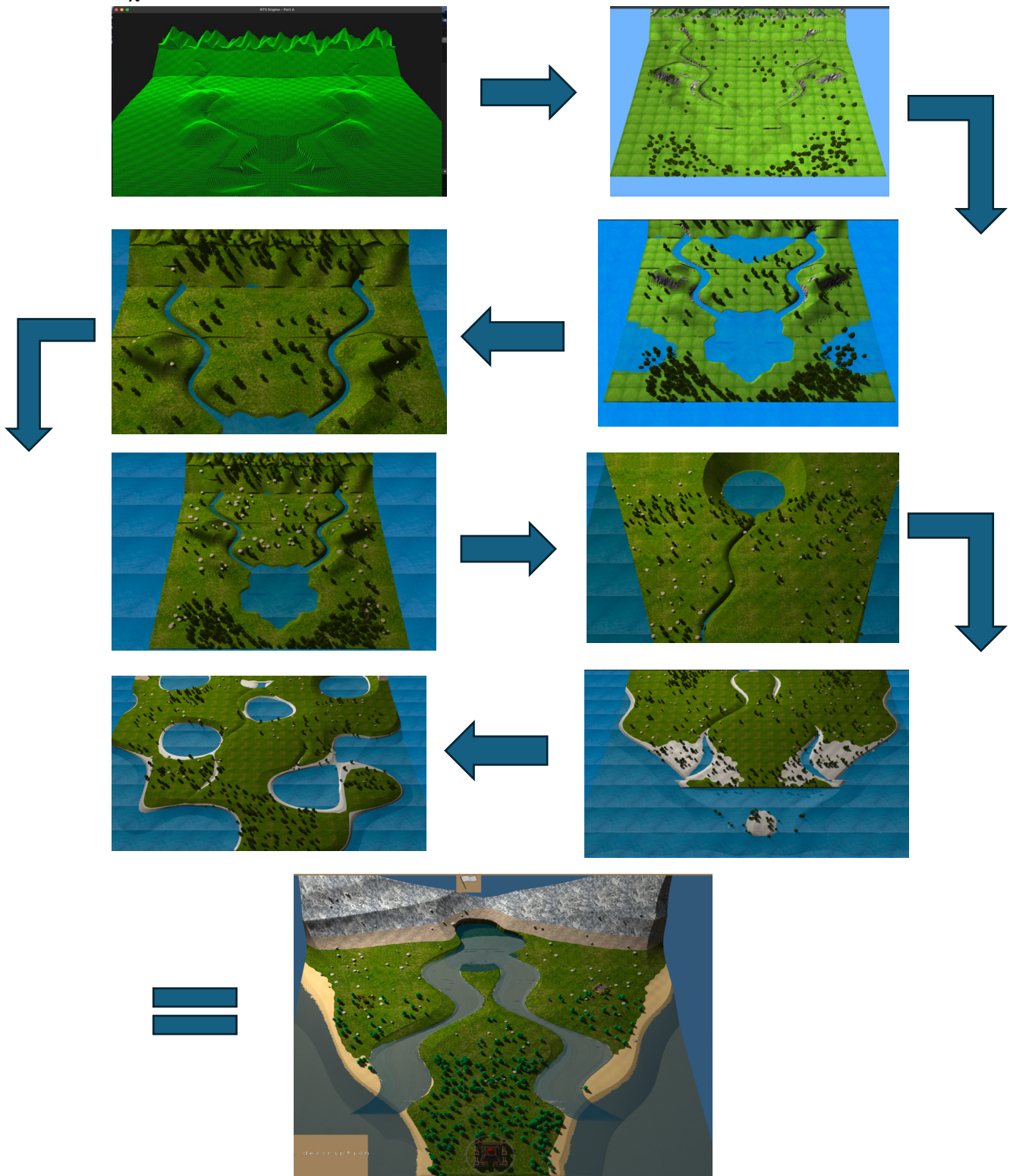
Μια σημαντική τεχνική δυσκολία ήταν η διαχείριση της στάθμης του νερού. Η απλοϊκή προσέγγιση "όπου το $Y < 0$ έχουμε νερό" αποδείχθηκε προβληματική.

- Το Πρόβλημα: Αν έθετα ένα ενιαίο επίπεδο νερού, θα πλημμύριζε αφύσικα περιοχές που ήθελα να είναι ξηρά αλλά χαμηλές, ή θα άφηνε τα ορεινά ποτάμια στεγνά αν ήταν ψηλότερα από το επίπεδο της θάλασσας. Ήθελα το έδαφος να είναι ανώμαλο και όχι μια επίπεδη πλάκα.
- Η Λύση: Αντί για ένα ενιαίο επίπεδο, υλοποίησα ξεχωριστά meshes για κάθε τύπο νερού:
 - Ένα mesh για τον Ωκεανό ($GenerateWaterGeometry$).

- Ένα ξεχωριστό mesh για τη Λίμνη (*generateLakeWater*) στο συγκεκριμένο ύψος lakeY.
- Ένα procedural mesh για τα Ποτάμια (*generateRiverWater*) που ακολουθεί ακριβώς τη χάραξη της κοίτης.

4. Προσωπική Ανασκόπηση

Ως το πρώτο κομμάτι που υλοποίησα, το έδαφος απαιτούσε τη μεγαλύτερη υπομονή. Η μετάβαση από τα μαθηματικά στην οπτική αναπαράσταση —το να κάνεις τους αριθμούς να μοιάζουν με βράχια και ακτές— ήταν μια διαδικασία συνεχούς trial and error. Ωστόσο, το αποτέλεσμα ενός χάρτη με φυσικά σύνορα το ακρωτήριο που χάνεται στη θάλασσα αντί για τεχνητούς τοίχους, προσέδωσε την αίσθηση "ελευθερίας" που ήθελα να αποπνέει ο κόσμος του παιχνιδιού.



1.2 Lighting/Shadow Algorithms

Αφού σμιλεύτηκε η γη, έπρεπε να λουστεί στο φως. Η αρχιτεκτονική του φωτισμού χωρίζεται σε δύο πράξεις: το Shadow Mapping και την τελική σύνθεση (Lighting Calculation).

1. Shadow Mapping

Πριν ζωγραφιστεί έστω και ένα pixel στην οθόνη του παίκτη, ο κώδικας εκτελεί ένα κρυφό πέρασμα: το DrawDepth. Εδώ, η κάμερα μεταφέρεται στη θέση του Ήλιου και κοιτάζει τον κόσμο κάθετα (lightSpaceMatrix).

- Η Πρόκληση της Κίνησης: Το δύσκολο δεν ήταν να ρίξω σκιές από στατικά κτίρια, αλλά από κινούμενους στρατιώτες. Στον Vertex Shader της σκιάς (shadow_depth.vert), ενσωμάτωσα τη λογική του Skinning (uBoneTexture) και του Instancing. Αυτό σημαίνει πως όταν ένας στρατιώτης περπατάει, η σκιά του κινείται και παραμορφώνεται ακριβώς όπως το σώμα του, χωρίς να χρειάζεται ο CPU να υπολογίζει κάθε τρίγωνο χωριστά.
- Η Κατασκευή: Ακόμα και τα ημιτελή κτίρια ρίχνουν σκιές που αντιστοιχούν στην πρόοδο κατασκευής τους (uBuildProgress), καθώς ο κώδικας επιλέγει δυναμικά ανάμεσα στο foundationModel και το finalModel.

2. Ambient, Diffuse, Specular

Στους Shaders του εδάφους και των μοντέλων (standard.frag, terrain.frag), το τελικό χρώμα προκύπτει από τον συνδυασμό τριών παραγόντων:

1. Το Περιβάλλον (Ambient): Ένα απαλό φως (0.3) που υπάρχει παντού, ώστε οι σκιές να μην είναι απόλυτα μαύρες.
2. Η Διάχυση (Diffuse): Το κύριο φως που αποκαλύπτει το σχήμα των αντικειμένων, υπολογισμένο με βάση τη γωνία που χτυπάει ο ήλιος την επιφάνεια ($\text{dot}(\text{norm}, \text{lightDir})$).
3. Η Αντανάκλαση (Specular): Η γυαλάδα στους βράχους. Χρησιμοποίησα το μοντέλο Phong ($\text{pow}(\dots, 16.0)$), το οποίο δημιουργεί έντονες φωτεινές κηλίδες εκεί που το φως αντανακλάται απευθείας στα μάτια του θεατή.

3. Σκιές (PCF)

Στον πραγματικό κόσμο, οι σκιές σπάνια είναι τόσο απότομες, για να αποφύγω λοιπόν "pixelated" αποτέλεσμα στις άκρες των σκιών, εφάρμοσα την τεχνική Percentage-Closer Filtering (PCF). Ο αλγόριθμος δεν ελέγχει απλώς ένα σημείο, αλλά παίρνει τον μέσο όρο ενός πλέγματος 3x3 γύρω από το pixel (ShadowCalculation), δημιουργώντας μια φυσική, απαλή μετάβαση από το φως στο σκοτάδι.

4. Το Φως στο Νερό: Fresnel και Αφρός

Στο νερό το φως δεν απορροφάται απλώς, αλλά διαθλάται και ανακλάται.

- Fresnel Effect: Υλοποίησα τον φυσικό νόμο του Fresnel ($\text{pow}(1.0 - \cos\Theta, 5.0)$), ώστε το νερό να είναι διαφανές όταν το κοιτάς κάθετα, αλλά να γίνεται καθρέφτης όταν το κοιτάς από μακριά.

- Λάμψη: Πρόσθεσα μια υφή θορύβου (sparkle) που κινείται με τον χρόνο, κάνοντας την επιφάνεια του ποταμού να στραφταλίζει κάτω από τον ψηφιακό ήλιο.
- Ο Αφρός: Αντί να ζωγραφίσω αφρό με το χέρι, τον υπολόγισα δυναμικά συγκρίνοντας το βάθος του νερού με το βάθος του βυθού. Όπου το νερό είναι ρηχό εμφανίζεται αυτόματα λευκός αφρός.



Εικόνα: Η λευκή γραμμή που εμφανίζεται στην εικόνα αντιστοιχεί σε εφέ αφρού. Σε επόμενη έκδοση του παιχνιδιού, με στόχο τη σταθεροποίηση και τη μείωση προβλημάτων που προέκυπταν από το αυξημένο rendering load, αφαιρέθηκαν δευτερεύουσες οπτικές λεπτομέρειες όπως το συγκεκριμένο εφέ και μειώθηκε η ποιότητα του νερού, ώστε να επιτευχθεί πιο ομαλή λειτουργία.

1.3 Το βλέμμα του αετού – Κάμερα RTS

Η μετάβαση στην πανοραμική θέα (Top-Down) ήταν απαραίτητη για τη μεταμόρφωση του παίκτη σε στρατηγό. Η διαδικασία αυτή βασίστηκε στον επαναπροσδιορισμό της ψηφιακής κάμερας, ώστε να προσφέρει πλήρη εποπτεία του πεδίου μάχης από μεγάλο υψόμετρο. Ο πρώτος άξονας αλλαγής αφορούσε το ύψος στον άξονα Y. Η κάμερα υψώθηκε δραστικά ώστε ο παίκτης να μπορεί να παρατηρεί τις κινήσεις των μονάδων σε μεγάλες αποστάσεις.

Η σημαντικότερη τροποποίηση έγινε στη γωνία κλίσης, την παράμετρο Pitch. Η τιμή ορίστηκε στις -90.0 μοίρες, μετατρέποντας την οπτική γωνία από διαγώνια σε απόλυτα κάθετη. Σε αυτή τη ρύθμιση, η κάμερα κοιτάζει το έδαφος σε ορθή γωνία, προσφέροντας την κλασική "στρατηγική" απεικόνιση όπου η οριζόντια περιστροφή (Yaw) παύει να επηρεάζει το οπτικό πεδίο.

Κεφάλαιο 2: Η Θεμελίωση του Βασιλείου

Λίθος επί λίθου και νόμος επί νόμου

2.1 GUI

Καθ' όλη τη διάρκεια της εργασίας, η στρατηγική μου επιλογή ήταν να ξεκινάω από την επιφάνεια και να προχωράω σταδιακά προς το βάθος. Πρώτα σχεδιάζοταν το GUI, οπτικοποιώντας την εμπειρία του παίκτη, και στη συνέχεια οικοδομούταν η αρχιτεκτονική του κώδικα που υποστηρίζει τις λειτουργίες στο παρασκήνιο.

Σχεδίασα το GUI ως το πρώτο βήμα, καθορίζοντας πώς ο παίκτης θα αλληλεπιδρά με το βασίλειό του.

- Πάνελ Κτιρίων και Παραγωγής: Δημιουργήθηκαν κουμπιά για την τοποθέτηση κτιρίων και την εκπαίδευση στρατιωτών, τα οποία εμφανίζονται δυναμικά ανάλογα με την επιλογή του παίκτη.
- Πληροφοριακά Πάνελ: Σχεδιάστηκαν πεδία που δείχνουν το κόστος κάθε ενέργειας και την κατάσταση των επιλεγμένων οντοτήτων.
- Μπάρα Πόρων: Μια σταθερή ένδειξη στο πάνω μέρος της οθόνης παρακολουθεί τα αποθέματα σε Τροφή, Ξύλο, Χρυσό, Μετάλλευμα και χωρητικότητα πληθυσμού.

2.2 Τα Θεμέλια: Κτίρια και Οντότητες

Ύστερα από αρκετή αναζήτηση στο διαδίκτυο βρήκα 3D Models animated και μη, τα οποία ικανοποιούσαν τους σκοπούς που είχα θέσει, αυτά ήταν τα εξής:

- Town Center & Villagers: Το Town Center παράγει Χωρικούς (economic unit) που συλλέγουν πόρους για την ανάπτυξη.
- Barracks & Combat Units: Οι Στρατώνες εκπαιδεύουν τον στρατό, προσφέροντας επιλογές για Archer (Ranged) και Knight (Melee).
- Εξειδικευμένα Κτίρια: Προστέθηκαν Φάρμες για τροφή, Αποθήκες για χωρητικότητα, Αγορά για αυτόματη παραγωγή χρυσού και Σπίτια για να είναι δυνατή η αύξηση του πληθυσμού.

2.3 Η Αρχιτεκτονική των Καταστάσεων (State Machine)

Για να μετατρέψω τα στατικά μοντέλα σε έξυπνες οντότητες, εφάρμοσα μια αυστηρή αρχιτεκτονική λογισμικού. Η συμπεριφορά των units δεν είναι τυχαία, αλλά διέπεται από μια Μηχανή Πεπερασμένων Καταστάσεων (FSM). Στον πυρήνα κάθε unit όρισα μια κεντρική μεταβλητή που λειτουργεί ως ο εγκέφαλός της. Αυτή η μεταβλητή υπαγορεύει σε ποια κατάσταση βρίσκεται το unit ανά πάσα στιγμή, απαγορεύοντας την ταυτόχρονη εκτέλεση αντικρουόμενων ενεργειών. Οι τέσσερις πυλώνες της ύπαρξής τους είναι:

- Αδράνεια (Idle): Η μονάδα περιμένει εντολές, σκανάροντας το περιβάλλον.
- Κίνηση (Moving): Η μονάδα εκτελεί τον αλγόριθμο εύρεσης μονοπατιού για να φτάσει σε έναν προορισμό.
- Εργασία (Gathering): Μια αποκλειστική κατάσταση για τους Χωρικούς, όπου εξορύσσουν πόρους.
- Μάχη (Combat): Η κατάσταση εμπλοκής με τον εχθρό, διαθέσιμη μόνο στους στρατιώτες.

Η αλλαγή κατάστασης γίνεται μέσω μεθόδου της Unit, η οποία δεν ενημερώνει μόνο την εσωτερική μεταβλητή κατάστασης, αλλά συγχρονίζει και το animation layer, εξασφαλίζοντας ότι η οπτική αναπαράσταση συμβαδίζει με τη λογική συμπεριφορά. Η λογική ενημέρωση των καταστάσεων εκτελείται κεντρικά στο game loop (κυρίως μέσω των Scene_Events update συναρτήσεων), όπου κάθε frame αξιολογούνται οι συνθήκες και γίνονται οι κατάλληλες μεταβάσεις.

2.4 Η λογική συλλογής των πόρων

Η οικονομική συμπεριφορά των Χωρικών δεν είναι μια απλή κίνηση, αλλά μια σειρά λογικών βημάτων που εκτελούνται κυκλικά:

- Ο Εντοπισμός (Spatial Query): Όταν ο παίκτης δώσει εντολή συλλογής, ο κώδικας δεν στέλνει τον εργάτη στα τυφλά. Σαρώνει μαθηματικά όλες τις θέσεις των δέντρων και των βράχων στον χάρτη για να εντοπίσει ποιος πόρος βρίσκεται στην μικρότερη Ευκλείδεια απόσταση από τον εργάτη.
- Η Χρονομέτρηση: Μόλις ο εργάτης φτάσει στον στόχο, ο αλγόριθμος ενεργοποιεί ένα εσωτερικό χρονόμετρο. Για κάθε δευτερόλεπτο που περνάει, προστίθεται πρόοδος στην εργασία.
- Η Ανταμοιβή: Μόλις το χρονόμετρο συμπληρώσει τον απαιτούμενο κύκλο (2 δευτερόλεπτα), ο κώδικας εκτελεί δύο ενέργειες ταυτόχρονα: αφαιρεί το αντικείμενο του πόρου (δέντρο/πέτρα) από τον τρισδιάστατο κόσμο και πιστώνει αυτόματα το αντίστοιχο υλικό στο ταμείο του παίκτη.

Αν ο worker έχει ενεργοποιημένο auto-gather τότε μετά την ολοκλήρωση ενός resource, γίνεται νέα αναζήτηση σε μεγαλύτερη ακτίνα και δημιουργείται αυτόματα νέο GatherTask. Η επιλογή γίνεται με greedy κριτήριο (πλησιέστερο resource).

2.5 Ο Πόλεμος

Η μάχη είναι ένας βρόχος ελέγχου που τρέχει συνεχώς για κάθε μονάδα. Εδώ η τεχνική πρόκληση ήταν να ξεχωρίσω πώς συμπεριφέρεται ένας Ιππότης από έναν Τοξότη, χρησιμοποιώντας Run-Time Type Information (RTTI):

- Άμεση Επαφή (Melee Logic): Για τους Ιππότες, ο κώδικας ελέγχει συνεχώς την απόσταση από τον στόχο. Αν ο εχθρός βρεθεί εντός εμβέλειας, ο αλγόριθμος αφαιρεί απευθείας Health Points από τον στόχο και ενεργοποιεί ένα cooldown πριν το επόμενο χτύπημα. Για την αποφυγή ταλαντώσεων στο όριο της εμβέλειας, χρησιμοποιείται μικρό περιθώριο ασφαλείας (π.χ. 90% της ακτίνας επίθεσης)
- Projectiles (Ranged Logic): Για τους Τοξότες, η ζημιά δεν είναι άμεση. Ο κώδικας δημιουργεί projectiles μια εντελώς νέα οντότητα στον κόσμο, ένα βέλος ή μια μπάλα φωτιάς. Αυτό το αντικείμενο αποκτά τη δική του φυσική υπόσταση και κινείται στον χώρο μέχρι να συγκρουστεί με τον στόχο, προσθέτοντας ρεαλισμό καθώς η επίθεση έχει χρόνο πτήσης.

2.6 Selection Ring

Για να ενημερώνεται ο παίκτης για την κατάσταση των units σχεδίασα ένα Selection Ring κάτω από κάθε unit, ο οποίος αλλάζει χρώμα δυναμικά. Ο κώδικας ελέγχει σε ποια κατάσταση βρίσκεται το κάθε unit. Εάν εργάζεται ή πολεμάει, στέλνει μια εντολή στον Shader να "βάλει" τον δακτύλιο σκούρο πράσινο, ενώ αν η μονάδα είναι σε αναμονή, η εντολή αλλάζει το χρώμα σε ελαφρύ πράσινο. Αυτή η αλλαγή δεν είναι μια απλή εναλλαγή εικόνων, αλλά μια μαθηματική ανάμειξη χρωμάτων (tinting).

2.7 Animation

Η Unit υποστηρίζει δύο επίπεδα animation: Base locomotion layer (idle/walk), Action layer (attack/gather). Όταν υπάρχει ενεργό action animation αυτό έχει προτεραιότητα και παρακάμπτει προσωρινά το locomotion animation. Όταν ολοκληρωθεί η ενέργεια καθαρίζεται το action animation και επανέρχεται στο locomotion state επιτρέποντας καθαρό διαχωρισμό μεταξύ state logic και animation logic.

Κεφάλαιο 3: Έλεγχος του Βασιλείου

Το αόρατο χέρι που ορίζει τη μοίρα των θνητών

3.1 Μηχανισμός Κάμερας Πρώτου/Τρίτου Προσώπου

Η λειτουργία αλλαγής κάμερας υλοποιήθηκε ως εναλλακτικό camera mode εντός της κλάσης Scene, επιτρέποντας στον παίκτη να παρακολουθεί τον κόσμο από την οπτική γωνία επιλεγμένης μονάδας.

1. Αρχιτεκτονική Κατάστασης Κάμερας

Το σύστημα διατηρεί:

- unitCameraActive_ → flag ενεργοποίησης,
- unitCameraTarget_ → pointer στο unit, savedCameraPos_,
- savedCameraYaw_, savedCameraPitch_ → αποθήκευση προηγούμενης κατάστασης ώστε να επιστρέψει μετά,
- unitCameraYawOffset_, unitCameraPitchOffset_ → σχετικές γωνιακές μετατοπίσεις

Η μετάβαση γίνεται μέσω της toggleUnitCamera() όπου αν η λειτουργία είναι ανενεργή τότε αποθηκεύεται η ελεύθερη (RTS) κάμερα, επιλέγεται το πρώτο unit από το selectedUnits_ και ενεργοποιείται το follow-mode. Αν γίνει ενεργή επαναφέρεται η αποθηκευμένη κάμερα και μηδενίζονται τα offsets.

2. Υπολογισμός Θέσης Κάμερας

Η ενημέρωση γίνεται ανά frame μέσω της updateUnitCameraView() όπου η κάμερα τοποθετείται σε σφαιρικές συντεταγμένες γύρω από το unit. Θεωρούμε r (ακτίνα απόστασης), ϕ (yaw offset), θ (pitch offset), προκύπτουν λοιπόν οι καρτεσιανές συντεταγμένες του offset:

$$\begin{aligned}x &= r \cos(\theta) \sin(\phi) \\y &= r \sin(\theta) \\z &= r \cos(\theta) \cos(\phi)\end{aligned}$$

Η τελική θέση κάμερας:

$$\vec{C} = \vec{P}_{unit} + \overrightarrow{offset}$$

και ο προσανατολισμός ορίζεται μέσω του LookAt($\vec{P}_{unit}$), με το pitch να περιορίζεται σε $\theta \in [-35^\circ, +25^\circ]$

3.2 Κατασκευή Κτιρίου (Gradual Construction)

Ακολουθήθηκε προσέγγιση διπλού mesh με γραμμική παρεμβολή της αδιαφάνειας, έχοντας κανονικοποιημένη πρόοδος $t = \frac{\text{elapsed time}}{\text{buildTime}}$. Το α λοιπόν στα θεμέλια υπολογίζεται $\alpha = 1 - t$, μέχρι το Final mesh: $\alpha = t$.

Η απόδοση γίνεται στο `Building::Draw()` μέσω shader uniform `uAlpha` και ενεργοποιημένου OpenGL blending. Το σύστημα επεκτείνεται και σε upgrade transitions με ίδια μαθηματική λογική.

3.3 Ray Casting

Για να μπορέσει ο παίκτης να δώσει διαταγές, πρέπει να υπάρχει ένας τρόπος να μεταφραστεί η δισδιάστατη κίνηση του ποντικιού στον τρισδιάστατο χώρο της μάχης. Αυτή η γέφυρα επικοινωνίας χτίστηκε πάνω στις λειτουργίες Ray Casting για την ακρίβεια και το Drag-Box για το πλήθος. Η διαδικασία ξεκινά με την αντιστροφή της πραγματικότητας. Μετατρέπουμε τις συντεταγμένες της οθόνης (NDC) σε συντεταγμένες του κόσμου, χρησιμοποιώντας τον αντίστροφο πίνακα της προβολής και της προοπτικής:

$$World = (P \cdot V)^{-1} \cdot NDC \quad \text{όπου } P = \text{projection matrix } V = \text{view matrix}$$

Επειδή όμως το έδαφός μας δεν είναι επίπεδο αλλά ανώμαλο η απλή τομή με ένα επίπεδο δεν αρκεί οπότε υλοποίησα ray marching:

$$P(d) = O + d \cdot \vec{D} \quad \text{μέχρι } P_y \leq H(x, z)$$

Ο έλεγχος σταματά τη στιγμή που η ακτίνα τρυπάει το έδαφος, δηλαδή όταν το ύψος της ακτίνας P_y γίνει μικρότερο ή ίσο από το ύψος του εδάφους $H(x, z)$ στο συγκεκριμένο σημείο.

3.4 Drag Box Selection

Όταν η μάχη απαιτεί τη μετακίνηση πολλών στρατιωτών η επιλογή γίνεται μαζική. Εδώ, αντί να φέρουμε το κλικ στον 3D κόσμο, κάνουμε το αντίστροφο: φέρνουμε τον 3D κόσμο στην 2D οθόνη. Η υλοποίηση γίνεται σε δύο στάδια:

1. Projection: Προβολή κάθε μονάδας:

$$screen = worldToScreen(worldPos)$$

2. Έλεγχος AABB (Axis-Aligned Bounding Box): Ελέγχουμε αν οι 2D συντεταγμένες της μονάδας βρίσκονται μέσα στο ορθογώνιο που σχεδίασε ο παίκτης, χρησιμοποιώντας απλές ανισότητες:

$$\begin{aligned} x_{min} &\leq x \leq x_{max} \\ y_{min} &\leq y \leq y_{max} \end{aligned}$$

Χρονική πολυπλοκότητα της κίνησης αυτή είναι $O(n)$, καθώς απαιτείται μόνο ένα πέρασμα από τη λίστα των units για να διαχωριστούν οι selected από τους υπόλοιπους.

Κεφάλαιο 4: Οδός - Pathfinding: Grid-based A* με δυναμική ενημέρωση εμποδίων

Κάθε βήμα χαραγμένο, κάθε διαδρομή προγεγραμμένη

4.1 Αρχιτεκτονική συστήματος πλοήγησης

Το σύστημα πλοήγησης του παιχνιδιού βασίζεται σε δισδιάστατο πλέγμα πλοήγησης, το οποίο καλύπτει ολόκληρη την επιφάνεια του terrain και διατηρείται σε επίπεδο Scene. Κάθε κελί του πλέγματος αποθηκεύεται σε γραμμικό πίνακα τύπου `std::vector<uint8_t> navWalkable_` και κωδικοποιεί την κατάσταση διέλευσης με τρεις δυνατές τιμές: πλήρως βατό (Walkable), μη βατό (Unwalkable) και υψηλού κόστους (HighCost).

Ο υπολογισμός διαδρομής πραγματοποιείται σε επίπεδο Scene για κάθε εντολή μετακίνησης μονάδας, μέσω της μεθόδου `Scene::findPath()`. Η δυναμική τροποποίηση του terrain όπως η δημιουργία κρατήρων, ενημερώνει το navigation grid μέσω της κλάσης `AStarGridUpdater`, εξασφαλίζοντας ότι η πλοήγηση αντανακλά την τρέχουσα γεωμετρία του περιβάλλοντος.

4.2 Δημιουργία πλέγματος και προϋπολογισμός εμποδίων

Οι διαστάσεις του πλέγματος υπολογίζονται με βάση το μέγεθος του terrain και την παράμετρο `navCellSize_`, όπως υλοποιείται στο αρχείο `Scene_Pathfinding.cpp`. Κατά τη δημιουργία του grid, κάθε κελί αξιολογείται ως προς την καταλληλότητα διέλευσης:

- Περιοχές νερού εντοπίζονται μέσω της συνάρτησης `isWaterArea()` και χαρακτηρίζονται ως μη βατές.
- Κτίρια εισάγονται ως κυκλικά εμπόδια μέσω της `markObstacleDisc()`, με ακτίνα που εξαρτάται από τον τύπο του κτιρίου.

Με αυτόν τον τρόπο, το αρχικό grid αποτελεί μια προϋπολογισμένη αναπαράσταση των στατικών εμποδίων του χάρτη.

4.3 Υλοποίηση αλγορίθμου A*

Η διαδικασία αναζήτησης διαδρομής ενεργοποιείται από τη ροή `Scene::commandUnitTo()` προς `Scene::findPath()`. Οι συντεταγμένες κόσμου μετατρέπονται σε δείκτες grid μέσω της `worldToNav()`, η οποία διασφαλίζει ότι οι τιμές παραμένουν εντός ορίων. Η υλοποίηση του A* χρησιμοποιεί τις κλασικές δομές δεδομένων όπως:

- πίνακα `gScore` για το κόστος από την αρχική θέση,
- πίνακα `cameFrom` για ανακατασκευή διαδρομής,
- σύνολο κλειστών κόμβων (closed set),
- ουρά προτεραιότητας με κριτήριο το άθροισμα κόστους και ευρετικής ($f = g + h$).

Το grid θεωρείται 8-axis, επιτρέποντας διαγώνια κίνηση με κόστος ίσο με $cellSize \times \sqrt{2}$. Η ευρετική συνάρτηση βασίζεται στην ευκλείδεια απόσταση μεταξύ τρέχοντος κόμβου και στόχου, προσφέροντας καλή ισορροπία μεταξύ ακρίβειας και υπολογιστικής απόδοσης.

Κελιά που χαρακτηρίζονται ως υψηλού κόστους δεν αποκλείονται πλήρως, αλλά επιβαρύνουν το κόστος μετάβασης. Συγκεκριμένα, όταν ένα κελί έχει κατάσταση HighCost, το βήμα κίνησης προς αυτό λαμβάνει αυξημένο κόστος, αποτρέποντας τη χρήση του εκτός αν δεν υπάρχει αποδοτικότερη εναλλακτική.

4.4 Απλοποίηση διαδρομής μέσω ελέγχου ορατότητας

Μετά την ολοκλήρωση της αναζήτησης και την ανακατασκευή της διαδρομής στο grid, εφαρμόζεται διαδικασία απλοποίησης. Ο στόχος είναι η μείωση του αριθμού ενδιάμεσων σημείων και η παραγωγή ομαλότερων διαδρομών. Η μέθοδος hasLineOfSight() εκτελεί έλεγχο ορατότητας μεταξύ δύο κόμβων με προσέγγιση τύπου Bresenham, σαρώνοντας τα ενδιάμεσα κελιά. Ένας greedy αλγόριθμος βασισμένος σε anchor-candidate λογική αφαιρεί ενδιάμεσα σημεία όταν υπάρχει απευθείας γραμμή ορατότητας, διατηρώντας μόνο τα απαραίτητα σημεία αλλαγής κατεύθυνσης.

4.5 Δυναμική ενημέρωση πλέγματος

Η κλάση AStarGridUpdater επιτρέπει την προσαρμογή του navigation grid σε πραγματικό χρόνο με βάση μεταβολές στο terrain. Για κάθε κόμβο δειγματοληπτείται η κλίση και το ύψος της επιφάνειας.

Κελιά με κλίση μεγαλύτερη από καθορισμένο όριο ή εντός πυρήνα κρατήρα χαρακτηρίζονται ως μη βατά, ενώ κελιά σε περιοχές με μέτρια κλίση ή στην περιφέρεια κρατήρα χαρακτηρίζονται ως υψηλού κόστους. Επιπλέον, μονάδες που βρίσκονται σε επηρεαζόμενα κελιά αναγκάζονται να επανυπολογίσουν διαδρομή, διασφαλίζοντας συνέπεια μεταξύ φυσικού περιβάλλοντος και πλοήγησης.

Κεφάλαιο 5: Αποδοτικότητα Rendering και Φυσικής: Frustum Culling και Instancing

Η τέχνη της αφαίρεσης

5.1 Αρχιτεκτονική Frustum Culling

Για τη βελτίωση της απόδοσης απόδοσης γραφικών, υλοποιείται frustum culling σε επίπεδο Scene. Σε κάθε frame εξάγονται τα επίπεδα του viewing frustum από τον πίνακα view-projection και εκτελούνται έλεγχοι ορατότητας με χρήση bounding spheres.

Η δομή Frustum, ορισμένη στο Scene.h, υπολογίζει τα επίπεδα του frustum από τον συνδυασμό προβολής και θέασης. Κάθε επίπεδο κανονικοποιείται ώστε οι υπολογισμοί απόστασης να είναι αριθμητικά σταθεροί.

Ο έλεγχος ορατότητας πραγματοποιείται μέσω της sphereInFrustum(center, radius), η οποία υπολογίζει την απόσταση της σφαίρας από κάθε επίπεδο. Αν η σφαίρα βρίσκεται πλήρως εκτός οποιουδήποτε επιπέδου, το αντικείμενο απορρίπτεται από το render pipeline.

5.2 Ακτίνα culling ανά τύπο οντότητας

Η μέθοδος `Scene::getEntityCullRadius()` επιστρέφει συντηρητική ακτίνα bounding sphere ανά τύπο οντότητας (μονάδες, κτίρια, props), λαμβάνοντας υπόψη τις γεωμετρικές διαστάσεις τους. Η χρήση συντηρητικών τιμών αποτρέπει οπτικά σφάλματα αποκοπής.

Το frustum culling εφαρμόζεται τόσο στο κύριο render pass όσο και στο shadow depth pass. Με τον τρόπο αυτό αποφεύγεται η επεξεργασία γεωμετρίας που δεν συμβάλλει στην τελική εικόνα ή στο shadow mapping.

5.3 Instancing για μεγάλο αριθμό αντικειμένων

Για στατικά αντικείμενα του περιβάλλοντος (π.χ. δέντρα και βράχοι) χρησιμοποιείται instanced rendering με στόχο τη μείωση των draw calls. Η μέθοδος `Model::DrawInstanced()` μεταφέρει πίνακα μετασχηματισμών μοντέλων σε instance buffer και ορίζει per-instance attributes με κατάλληλο divisor, ώστε κάθε instance να χρησιμοποιεί διαφορετικό μετασχηματισμό.

Η τεχνική instancing χρησιμοποιείται τόσο στο κύριο render pass όσο και στο depth pass για shadow mapping, επιτρέποντας την αποδοτική απόδοση μεγάλου πλήθους παρόμοιων αντικειμένων με μία μόνο κλήση σχεδίασης.

5.4 Περιορισμοί instancing για units

Τα units δεν αποδίδονται με instancing στην τρέχουσα υλοποίηση, λόγω της χρήσης skeletal animation. Κάθε unit διαθέτει μοναδικό σύνολο bone matrices, animation state και πιθανές διαφοροποιήσεις σε animation timing ή action states. Η υποστήριξη instancing για animated μονάδες θα απαιτούσε:

- μεταφορά bone matrices ανά instance (μέσω texture arrays ή SSBO),
- shader που επιλέγει κατάλληλη παλέτα οστών ανά instance,
- μηχανισμό batching units με κοινά animation clips και χρονισμό.

Στην παρούσα σχεδίαση, η απόδοση διασφαλίζεται μέσω frustum culling και ελεγχόμενου πλήθους units, ενώ το instancing περιορίζεται σε στατικά αντικείμενα του κόσμου όπου προσφέρει μέγιστο όφελος χωρίς επιπλέον πολυπλοκότητα.

Κεφάλαιο 6: Αποκάλυψις - Μηχανισμός “Explode on Command”: Από την εντολή χρήστη έως την έκρηξη

Όταν η γη τρέμει και η μορφή του κόσμου ραγίζει

6.1 Διαδρομή εντολής (UI → Καταστροφή μονάδας)

Η λειτουργικότητα “explode on command” δεν υλοποιείται ως ξεχωριστή ικανότητα, αλλά ως αποτέλεσμα της διαδικασίας διαγραφής μονάδας. Στο UI το πάνελ πληροφοριών του unit

ορίζει κουμπί Delete, του οποίου ο event handler είναι η μέθοδος `handleDeleteCurrentUnit()` (`Scene_UI.cpp`).

Η μέθοδος αυτή καλεί την κεντρική ρουτίνα `deleteUnit(unitInfoTarget_)`, η οποία είναι υπεύθυνη για την απομάκρυνση της μονάδας από τη σκηνή και την εκτέλεση όλων των συνοδευτικών ενεργειών (`Scene_UI.cpp`).

6.2 Ενεργοποίηση έκρηξης

Η `deleteUnit()` (`Scene_Events.cpp`) αποτελεί το ενιαίο σημείο χειρισμού της καταστροφής των units. Στο εσωτερικό της, πραγματοποιείται κλήση `UnitExplosionSystem`, το οποίο ενεργοποιεί έκρηξη στη θέση του unit. Οι παράμετροι της έκρηξης (ακτίνα, βάθος παραμόρφωσης, δύναμη) προσδιορίζονται δυναμικά μέσω της `getExplosionParamsForUnit`, εξασφαλίζοντας διαφοροποίηση ανά τύπο unit. Αυτό συνεπάγεται με μεγαλύτερη ακτίνα κρατήρα, μεγαλύτερο βάθος εδαφικής παραμόρφωσης, ισχυρότερη δυναμική επίδραση.

6.3 Αρχιτεκτονική συστήματος εκρήξεων (Queue-Based Processing)

Η μέθοδος `TriggerExplosion()` (`UnitExplosionSystem.cpp`) δεν εκτελεί άμεσα την παραμόρφωση. Αντίθετα, δημιουργεί ένα `ExplosionEvent` και το εισάγει σε εσωτερική ουρά. Η επιλογή αυτή εξυπηρετεί δύο στόχους:

- Αποσύνδεση λογικής εντολής από βαριά υπολογιστική εργασία.
- Έλεγχο απόδοσης, μέσω περιορισμού του πλήθους εκρήξεων ανά frame (σταθερά `kMaxExplosionsPerFrame = 2`).

Με αυτόν τον τρόπο αποφεύγονται `frame spikes`. Κατά την επεξεργασία της ουράς (`processExplosionQueue()`), κάθε συμβάν οδηγεί με τη σειρά σε δημιουργία κρατήρα, ενεργοποίηση οπτικών εφέ, και ενημέρωση του `navigation grid`.

6.4 Μόνιμη παραμόρφωση εδάφους

Η `applyCrater()` καλεί την `deformTerrainVertices()` η οποία:

- Διατρέχει το σύνολο των κορυφών του `terrain mesh`.
- Υπολογίζει την απόσταση κάθε κορυφής από το επίκεντρο.
- Εφαρμόζει τετραγωνική συνάρτηση απόσβεσης εντός της ακτίνας.
- Μειώνει το ύψος y της κορυφής ανάλογα με το βάθος και τον συντελεστή απόσβεσης.
- Περιορίζει την παραμόρφωση σε ελάχιστο βάθος (`bedrock clamp`).

Στη συνέχεια:

- Υπολογίζονται εκ νέου τα `normals` στην επηρεαζόμενη περιοχή.
- Συγχρονίζεται το `vertex buffer` με την GPU.

Η παραμόρφωση θεωρείται μόνιμη, καθώς τροποποιεί απευθείας το βασικό `vertex buffer` του `terrain` (`terrain->GetVertices()`), και όχι κάποιο προσωρινό `layer` ή `shader-based` εφέ. Επομένως, ο κρατήρας παραμένει στο περιβάλλον για όλη τη διάρκεια της εκτέλεσης.

6.5 Ένωση με Pathfinding

Μετά την παραμόρφωση, το σύστημα καλεί την `UpdateNavigationGridAfterDeformation`. Η μέθοδος αυτή ανανεώνει τα δεδομένα ύψους μέσω `terrainDeformer_ -> RefreshFromTerrain()` και ενημερώνει το grid μέσω `gridUpdater_ -> UpdateGridAfterDeformation(centerWorld, radiusWorld)`.

Η `AStarGridUpdater::UpdateGridAfterDeformation()` (`AStarGridUpdater.cpp`) πρακτικά εντοπίζει όλους τους κόμβους εντός της ακτίνας. Έπειτα, υπολογίζει κλίση και απόσταση από το επίκεντρο, κατηγοριοποιώντας κάθε κόμβο ως:

- Unwalkable, αν η κλίση υπερβαίνει το κρίσιμο όριο ή αν βρίσκεται στον πυρήνα του κρατήρα.
- HighCost, αν βρίσκεται σε ζώνη υψηλής κλίσης. Οι κόμβοι αυτοί παραμένουν προσπελάσιμοι, αλλά με αυξημένο κόστος.
- Walkable, σε αντίθετη περίπτωση.

Οι αλλαγές εγγράφονται στον πίνακα `navWalkable_`, που αποτελεί το υπόβαθρο του A*. Units που βρίσκονται σε επηρεαζόμενους κόμβους λαμβάνουν εντολή εκκαθάρισης στόχου κίνησης (`ClearMoveTarget()`), ώστε να ενεργοποιηθεί νέος υπολογισμός διαδρομής βάσει των ενημερωμένων δεδομένων. Στη διαδικασία αναζήτησης διαδρομής (`Scene_Pathfinding.cpp`), ο A* χρησιμοποιεί τον πίνακα `navWalkable_` για τον έλεγχο βατότητας. Ως αποτέλεσμα, η γεωμετρική παραμόρφωση μετατρέπεται, συνεπώς, σε λειτουργικό περιορισμό πλοήγησης.

Κεφάλαιο 7: Fog of War

Το Πέπλο της Άγνοιας

7.1 Σύστημα Fog of War

Το σύστημα Fog of War αποτελεί βασικό μηχανισμό στα παιχνίδια στρατηγικής πραγματικού χρόνου, καθώς εισάγει την έννοια της ατελούς πληροφορίας. Σε αντίθεση με ένα πλήρως ορατό πεδίο μάχης, ο παίκτης έχει πρόσβαση μόνο σε πληροφορία που παράγεται από τα units και τα κτήριά του.

Ο κόσμος του παιχνιδιού προβάλλεται σε ένα δισδιάστατο grid, το οποίο αντιστοιχεί στο navigation mesh του χάρτη. Κάθε κελί του πλέγματος αναπαριστά μία μικρή περιοχή του κόσμου και συνδέεται με μία κατάσταση ορατότητας. Η κατάσταση κάθε κελιού μπορεί να λάβει τρεις διακριτές τιμές:

- Μη εξερευνημένο (Unexplored)
- Εξερευνημένο αλλά όχι ορατό τη δεδομένη στιγμή (Explored)
- Πλήρως ορατό τη δεδομένη στιγμή (Visible)

Η επιλογή τριών καταστάσεων αντί δύο επιτρέπει την αποθήκευση ιστορικής πληροφορίας. Ο παίκτης διατηρεί γνώση του χάρτη ακόμη και όταν δεν έχει άμεση οπτική επαφή με μία περιοχή.

7.2 Υπολογισμός Ορατότητας

Η ορατότητα υπολογίζεται δυναμικά σε κάθε χρονικό βήμα της προσομοίωσης. Κάθε μονάδα και κάθε κτήριο διαθέτει ακτίνα ορατότητας, η οποία εξαρτάται από τον τύπο του αντικειμένου. Για κάθε οντότητα, ορίζεται ένας κυκλικός δίσκος ορατότητας στο οριζόντιο επίπεδο. Ένα κελί του πλέγματος θεωρείται ορατό εάν το κέντρο του βρίσκεται εντός της ακτίνας αυτής.

Μαθηματικά, αν $C = (x_c, z_c)$ είναι η θέση της οντότητας και $P = (x, z)$ το κέντρο του κελιού, τότε το κελί είναι ορατό εάν:

$$(x - x_c)^2 + (z - z_c)^2 \leq r^2$$

όπου r η ακτίνα ορατότητας. Η χρήση του τετραγώνου της απόστασης αποφεύγει τον υπολογισμό τετραγωνικής ρίζας και μειώνει το υπολογιστικό κόστος.

7.3 Απόδοση και Οπτική Αναπαράσταση

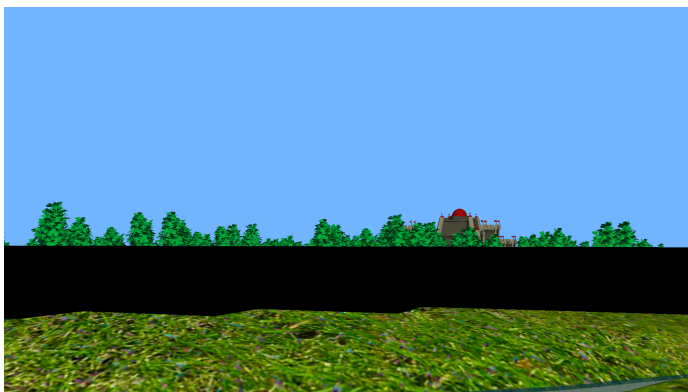
Η οπτική υλοποίηση γίνεται μέσω επιπέδου επικάλυψης (overlay) που τοποθετείται πάνω από τον χάρτη. Για κάθε κελί δημιουργείται γεωμετρικό τετράπλευρο, το οποίο λαμβάνει διαφορετική τιμή διαφάνειας ανάλογα με την κατάσταση:

- Πλήρως αδιαφανές για μη εξερευνημένες περιοχές.
- Ημιδιαφανές για εξερευνημένες.
- Καμία σχεδίαση για πλήρως ορατές περιοχές.

Η τεχνική αυτή βασίζεται σε alpha blending και επιτρέπει ομαλή ενσωμάτωση του fog με τη σκηνή χωρίς να επηρεάζεται το σύστημα φωτισμού.



Παρατηρούμε στη πάνω εικόνα τη λογική του explored, unexplored, visible και μη.



Στη κάτω εικόνα παρατηρούμε πως φαίνεται μέσα από first person camera το overlay που έχει χρησιμοποιηθεί ως το fog of war

Κεφάλαιο 8: Capture the Flag

Το παιχνίδι του θρόνου

8.1 Βασικό Σύστημα

Ο μηχανισμός Capture the Flag υλοποιεί αντικειμενικό στόχο νίκης βασισμένο στον χωρικό έλεγχο μίας περιοχής. Σε αντίθεση με την πλήρη εξόντωση του αντιπάλου, η νίκη επιτυγχάνεται μέσω διατήρησης παρουσίας σε συγκεκριμένο σημείο για προκαθορισμένο χρονικό διάστημα. Ο μηχανισμός αυτός ενθαρρύνει συγκρούσεις σε κεντρικά σημεία, δημιουργεί δυναμική ροή μάχης και αποτρέπει παθητικό παιχνίδι.

8.2 Μοντελοποίηση στον χώρο

Στο κέντρο χαμηλά του χάρτη τοποθετείται ένα monument-target. Γύρω από αυτό ορίζεται κυκλική ζώνη ακτίνας R , όπου το unit θεωρείται ότι συμμετέχει στην κατάληψη εάν η θέση του βρίσκεται εντός της ζώνης αυτής.

Η κατάσταση κατάληψης περιγράφεται από συνεχή μεταβλητή $p \in [0,1]$, όπου:

- $p = 0$ αντιστοιχεί σε ουδέτερη κατάσταση.
- $p = 1$ σημαίνει πλήρη κατάληψη.

Ο χρόνος που απαιτείται για πλήρη κατάληψη είναι T . Όταν $p = 1$, ενεργοποιείται άμεσα συνθήκη νίκης για τον αντίστοιχο παίκτη. Η νίκη βασίζεται αποκλειστικά στη χρονική διατήρηση χωρικού ελέγχου και όχι στην αριθμητική υπεροχή μονάδων εκτός περιοχής.

8.3 Δυναμικό Μοντέλο

Η εξέλιξη της προόδου κατάληψης ακολουθεί διαφορική εξίσωση πρώτης τάξης:

$$\frac{dp}{dt} = \begin{cases} \frac{1}{T} & \text{όταν μόνο ένας παίκτης βρίσκεται εντός περιοχής} \\ 0 & \text{όταν και οι δύο βρίσκονται εντός} \\ -\frac{2}{T} & \text{όταν κανένας δεν βρίσκεται εντός} \end{cases}$$

Η πρόοδος αυξάνεται γραμμικά όταν ένας παίκτης ελέγχει αποκλειστικά τη ζώνη. Σε περίπτωση ταυτόχρονης παρουσίας, η διαδικασία παγώνει. Σε πλήρη απουσία, η πρόοδος μειώνεται ταχύτερα ώστε να αποφεύγονται μόνιμα ημιτελείς καταστάσεις. Η πρόοδος κατάληψης αποδίδεται μέσω κυκλικής γραμμικής ένδειξης (radial progress indicator), η οποία αυξάνεται αναλογικά με την τιμή p . Το χρώμα της ένδειξης μεταβάλλεται δυναμικά ώστε να υποδηλώνει τον ενεργό παίκτη.



Κεφάλαιο 9: Συνοπτική Ανάλυση Αρχιτεκτονικής Κώδικα και Δομής Έργου

9.1 Δομή Εργασίας

Η αρχιτεκτονική του έργου ακολουθεί την αρχή του διαχωρισμού ευθυνών. Ο πηγαίος κώδικας οργανώνεται σε διακριτές θεματικές ενότητες, καθεμία από τις οποίες εξυπηρετεί συγκεκριμένο ρόλο στο σύστημα. Η κύρια δομή του καταλόγου πηγαίου κώδικα περιλαμβάνει τις ακόλουθες ενότητες:

- core/: Κεντρικός μηχανισμός σκηνής (scene engine), rendering, pathfinding και βασική διαχείριση UI.
- game/: Ορισμοί και υλοποιήσεις οντοτήτων παιχνιδιού (μονάδες, κτήρια).
- network/: Υλοποίηση multiplayer λειτουργίας μέσω τοπικού δικτύου (LAN) και διαχείριση πρωτοκόλλου επικοινωνίας.
- vfx/: Οπτικά εφέ (εκρήξεις, βλήματα, σύστημα σωματιδίων).
- audio/: Διαχείριση ηχητικών πόρων.
- gui/: Γραφικά στοιχεία διεπαφής (widgets, μενού, κουμπιά).
- raycast/: Μηχανισμός επιλογής αντικειμένων
- easteregg/: Προαιρετική λειτουργία στο βουνό (μηχανισμός πλημμύρας).
- rendering/: Βοηθητικές δομές και εργαλεία απόδοσης γραφικών.

Παράλληλα, υπάρχουν δύο επιπλέον βασικοί κατάλογοι:

- common/: Κοινές βιβλιοθήκες και βοηθητικές κλάσεις (Shader, Model, Texture, Noise), οι οποίες χρησιμοποιούνται από πολλαπλές ενότητες.
- external/: Εξωτερικές βιβλιοθήκες τρίτων (GLFW, GLEW, GLM, Assimp κ.ά.).

Η επιλογή οργάνωσης ανά λειτουργικότητα και όχι ανά τύπο αρχείου (π.χ. όλα τα headers μαζί) επιτρέπει ευκολότερη πλοήγηση στον κώδικα και μείωση αλληλεξαρτήσεων

9.2 Ποσοτική Ανάλυση Κώδικα

Το σύστημα περιλαμβάνει συνολικά περίπου 13.500 γραμμές κώδικα (αρχεία πηγαίου κώδικα και επικεφαλίδων). Ο φάκελος core/ αποτελεί τον πυρήνα του συστήματος και συγκεντρώνει περίπου το 57% του συνολικού κώδικα (~7.630 γραμμές). Περιλαμβάνει τον βασικό βρόχο παιχνιδιού, τη διαχείριση μάχης και συλλογής πόρων, το rendering pipeline και τη διαχείριση σκίων και Fog of War.

Ο φάκελος common/ αντιστοιχεί περίπου στο 9% του συνολικού κώδικα (~1.163 γραμμές). Περιλαμβάνει γενικής χρήσης κλάσεις για φόρτωση και διαχείριση τρισδιάστατων μοντέλων με animation (δηλαδή για αρχεία gltf, fbx κλπ). Η συγκεκριμένη δουλειά αποτέλεσε ιδιαίτερα απαιτητική και χρειάστηκε αρκετό troubleshooting ώστε να βρεθεί σωστός τρόπος φόρτωσης αυτών των αρχείων, ο οποίος καλύπτει όλες τις πιθανές ιδιαιτερότητες τους. Ακόμη βοηθάει στη διαχείριση shaders OpenGL και στη παραγωγή θορύβου για διαδικασιακή δημιουργία terrain.

Ο φάκελος VFX αντιστοιχεί περίπου στο 7% του συνολικού έργου (~1.027 γραμμές). Αποτελεί αυτόνομο σύστημα, το οποίο διαχειρίζεται τις εκρήξεις των units, υλοποιεί projectiles και particle system.

Ο φάκελος game περιέχει την λογική των entities, δηλαδή κλάσεις units και κτηρίων αντιστοιχούν περίπου στο 10% του συνολικού κώδικα (~1.400 γραμμές). Η σχεδίαση ακολουθεί αντικειμενοστραφή προσέγγιση, με διακριτές κλάσεις για:

- Πολεμικές μονάδες (π.χ. τοξότες, ιππότες).

- Economic units.
- Κτήρια (Town Center, Farm κ.λπ.).

Στα RTS παιχνίδια συνηθίζεται μια λογική ECS η οποία βασίζεται στο composition over inheritance. Αυτή η λογική είναι ιδιαίτερα χρήσιμη γιατί αποφεύγει diamond problems, σε πολύπλοκα συστήματα και κάνει ταχύτερη την λογική του κώδικα καθώς κάθε unit αποτελεί ξεχωριστό entity από μόνο του με ιδιότητες. Προσπάθησα να την υλοποιήσω κατά την διάρκεια των Χρριστουγέννων αλλά συνειδητοποίησα πως για το μέγεθος του project δεν θα είχε βελτιστοποιήση στην απόδοση του συστήματος και απλά θα χρειαζόταν να αλλάξω αρκετά το game logic.

Ο φάκελος network αντιστοιχεί περίπου στο 5% του συνολικού έργου (~650 γραμμές), περιλαμβάνοντας TCP wrapper για διαχείριση σύνδεσης και πρωτόκολλο ανταλλαγής εντολών παιχνιδιού. Η λογική του δικτύου ενσωματώνεται άμεσα στο κεντρικό state της σκηνής, καθώς κάθε ενέργεια παιχνιδιού (κίνηση, επίθεση, συλλογή) έχει αντίστοιχη δικτυακή αναπαράσταση.

9.3 Διαχείριση Μεγάλων Αρχείων και Συνεκτικότητα

Το μεγαλύτερο αρχείο του έργου SceneEvents.cpp (~2.777 γραμμές) συγκεντρώνει τις βασικές λειτουργίες του game loop. Παρά το μέγεθός του, η διατήρησή του ως ενιαίο αρχείο αποτελεί συνειδητή σχεδιαστική επιλογή.

Ιδιαίτερη σημασία δόθηκε στη δομική οργάνωση των αρχείων του έργου, με στόχο την αποφυγή δημιουργίας υπερβολικά μεγάλων και δυσανάγνωστων μονάδων πηγαίου κώδικα. Το αρχείο που παρουσιάζεται παραπάνω αποτελεί συνειδητή εξαίρεση και όχι τον κανόνα της αρχιτεκτονικής προσέγγισης.

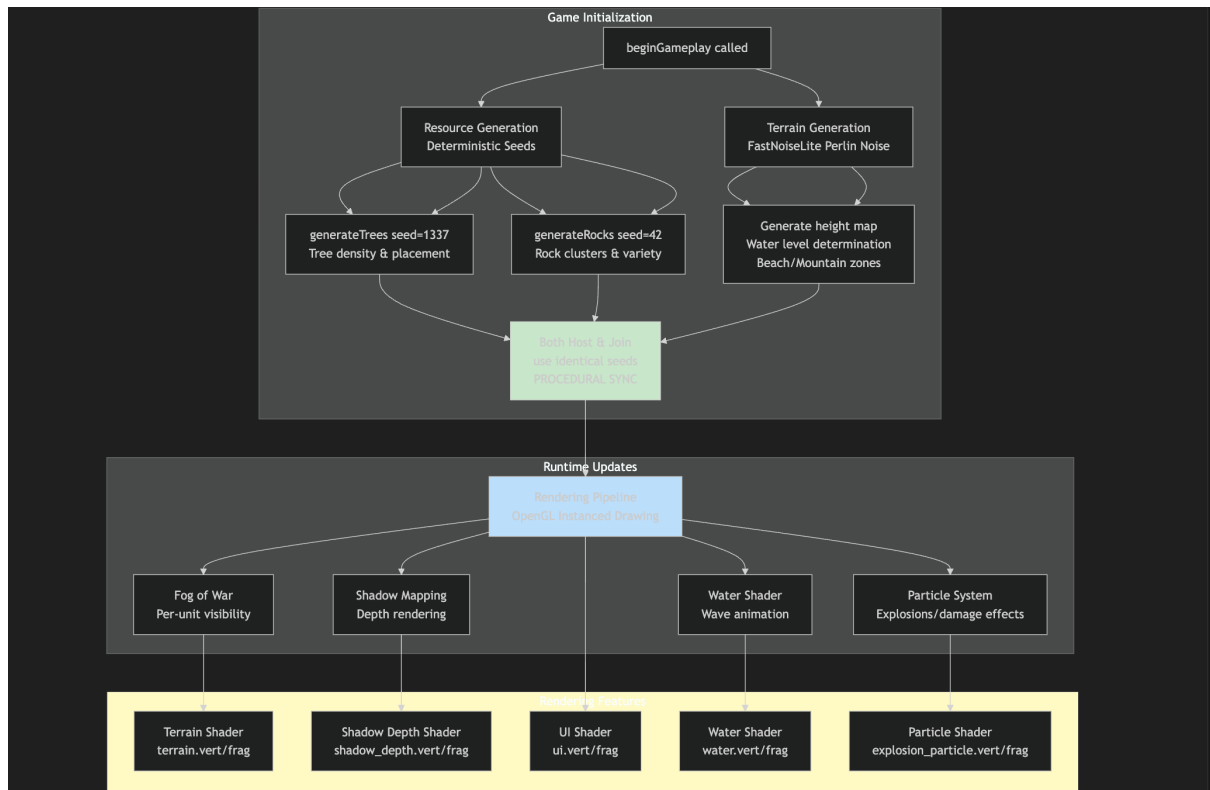
Στο αρχικό στάδιο ανάπτυξης, ο κεντρικός χειρισμός της σκηνής υλοποιούνταν μέσω ενός ενιαίου αρχείου το οποίο συγκέντρωνε όλες τις σχετικές λειτουργίες (αρχικοποίηση, ενημέρωση λογικής, απόδοση, διαχείριση γεγονότων κ.λπ.). Η πρακτική αυτή, αν και λειτουργική, οδηγούσε σε αυξημένη πολυπλοκότητα και δυσχέρεια συντήρησης.

Για τον λόγο αυτό αφιερώθηκε ξεχωριστό στάδιο αναδιοργάνωσης του κώδικα, κατά το οποίο πραγματοποιήθηκε συστηματική διάσπαση της μονολιθικής υλοποίησης σε επιμέρους αρχεία με σαφώς ορισμένες ευθύνες. Συγκεκριμένα, το αρχικό ενιαίο αρχείο σκηνής αναδομήθηκε σε διακριτές ενότητες, όπως:

- Scene_Init, υπεύθυνο για την αρχικοποίηση και τη δημιουργία αντικειμένων,
- Scene_Events, υπεύθυνο για τον κεντρικό βρόχο ενημέρωσης και τη διαχείριση γεγονότων,
- επιπλέον αρχεία για rendering, UI και επιμέρους συστήματα.

Παράλληλα, διατηρήθηκε ένα μικρό και λιτό αρχείο main, το οποίο περιορίζεται αποκλειστικά στην εκκίνηση του προγράμματος και στη μεταβίβαση ελέγχου στο σύστημα σκηνής. Με τον τρόπο αυτό επιτεύχθηκε σαφής διαχωρισμός μεταξύ σημείου εισόδου της εφαρμογής και λειτουργικής λογικής του παιχνιδιού.

Η αναδιάρθρωση δεν πραγματοποιήθηκε αποσπασματικά, αλλά ως συνειδητή σχεδιαστική απόφαση με στόχο την ενίσχυση της ποιότητας λογισμικού και την αποφυγή τεχνικού χρέους σε μελλοντικά στάδια ανάπτυξης.



Σχήμα: Το διάγραμμα απεικονίζει τη διαδρομή από τη γένεση του κόσμου έως την τελική απεικόνιση. Το σύστημα ξεκινά με την ντετερμινιστική διαδικαστική παραγωγή πόρων και εδάφους, χρησιμοποιώντας κοινά seed για να διασφαλιστεί ο απόλυτος συγχρονισμός μεταξύ των παικτών. Στη συνέχεια, η σκυτάλη περνά στο Runtime, όπου το Rendering Pipeline αξιοποιεί την τεχνική Instancing για τη βελτιστοποίηση της απόδοσης, καταλήγοντας στους εξειδικευμένους Shaders (Terrain, Shadow, Water, Particle) που συνθέτουν την οπτική ταυτότητα του παιχνιδιού.

Κεφάλαιο 10: Διάδοση

Η δημιουργία ενός ζωντανού και διαδραστικού κόσμου ήταν μόνο το πρώτο μισό της διαδρομής. Το εξίσου σημαντικό κομμάτι ήταν να διασφαλίσω ότι αυτός ο κόσμος θα μπορεί να υπάρχει και να λειτουργεί απρόσκοπτα σε κάθε περιβάλλον, ανεξάρτητα από το λειτουργικό σύστημα του χρήστη. Παράλληλα, έπρεπε να οργανώσω τη διαδικασία ανάπτυξης με τέτοιο τρόπο ώστε καμία πρόοδος αλλά και κανένα λάθος να μη χαθεί.

1. Η Πρόκληση της Συμβατότητας (Cross-Platform Compatibility)

Η ταυτόχρονη υποστήριξη Windows και macOS μετέτρεψε την ανάπτυξη σε μια απαιτητική άσκηση προσαρμογής. Παρότι η C++ αποτελεί μια κοινή βάση, κάθε λειτουργικό σύστημα έχει τις δικές του ιδιαιτερότητες στη μεταγλώττιση, στη διαχείριση αρχείων και στη φόρτωση βιβλιοθηκών.

Ιδιαίτερη προσοχή δόθηκε στη δημιουργία και διαχείριση των OpenGL contexts μέσω GLFW, ώστε να λειτουργούν ομαλά και στα δύο περιβάλλοντα. Παράλληλα, προσαρμόστηκαν τα file

paths ώστε να είναι συμβατά τόσο με τη δομή αρχείων των Windows όσο και με το Unix-based σύστημα του macOS.

2. Release Management

Η μετάβαση από τον πηγαίο κώδικα σε ένα πλήρως λειτουργικό και διανεμήσιμο εκτελέσιμο αρχείο αποτέλεσε ξεχωριστή πρόκληση, κυρίως λόγω της διαχείρισης dependencies.

Windows (.exe):

Δημιούργησα Release builds βελτιστοποιημένα για απόδοση, αφαιρώντας debug πληροφορίες. Το βασικό ζήτημα ήταν η σωστή διαχείριση των DLLs (Dynamic Link Libraries). Για να εξασφαλίσω φορητότητα, τοποθέτησα όλα τα απαραίτητα αρχεία στον ίδιο φάκελο με το εκτελέσιμο, δημιουργώντας ένα πλήρως portable πακέτο που μπορούσε να εκτελεστεί σε οποιοδήποτε σύστημα χωρίς επιπλέον εγκαταστάσεις.

macOS (.app):

Στο macOS, η διαδικασία απαιτούσε τη δημιουργία ενός Application Bundle (.app). Επειδή το σύστημα δεν επιτρέπει εύκολα την εξωτερική φόρτωση δυναμικών βιβλιοθηκών, χρησιμοποίησα εργαλεία όπως το dylibbundler για να ενσωματώσω τις απαραίτητες βιβλιοθήκες εντός του bundle. Με αυτόν τον τρόπο, η εφαρμογή έγινε πλήρως αυτόνομη και συμβατή με τις απαιτήσεις του λειτουργικού.

3. Το Δίκτυο Ασφαλείας (Version Control & GitHub)

Σε όλη τη διάρκεια της ανάπτυξης, το GitHub λειτούργησε όχι απλώς ως αποθηκευτικός χώρος, αλλά ως πλήρες ιστορικό εξέλιξης του έργου και ως μηχανισμός ασφάλειας.

Καταγραφή Εξέλιξης:

Κάθε τροποποίηση στον κώδικα καταγραφόταν μέσω commits, δημιουργώντας μια σαφή και αναλυτική ακολουθία αλλαγών. Αυτό επέτρεψε την παρακολούθηση της προόδου και την τεκμηρίωση της ανάπτυξης.

Fallback & Ανάκαμψη:

Κατά τη διαδικασία πειραματισμού προέκυψαν στιγμές όπου νέες αλλαγές επηρέασαν αρνητικά τη σταθερότητα του παιχνιδιού. Η δυνατότητα άμεσης επαναφοράς (revert) σε μια προηγούμενη, σταθερή έκδοση αποδείχθηκε καθοριστική. Αυτό μου επέτρεψε να εξερευνώ νέες ιδέες χωρίς φόβο, γνωρίζοντας ότι οποιοδήποτε σφάλμα μπορούσε να αναιρεθεί και η ανάπτυξη να συνεχιστεί από ένα ασφαλές σημείο.



Στατιστικά χρήσης του github, πρέπει να είχα σύνολο 40+ commits

Επίλογος: Η Έβδομη Ημέρα

Και είδεν ότι καλόν

Κοιτάζοντας πίσω, από την πρώτη γραμμή κώδικα μέχρι το τελικό εκτελέσιμο αρχείο, η διαδρομή αυτή ήταν κάτι περισσότερο από μια ακαδημαϊκή υποχρέωση. Ήταν μια διαδικασία ωρίμανσης. Ξεκινώντας με μαθηματικούς τύπους για την παραγωγή του εδάφους, φτάσαμε στη δημιουργία ενός κόσμου που «αναπνέει», όπου οι οντότητες δεν εκτελούν απλώς εντολές, αλλά αντιλαμβάνονται το περιβάλλον τους, χαράζουν μονοπάτια και μάχονται για την επικράτηση.

Η μεγαλύτερη ικανοποίηση δεν προήλθε από την επίλυση των πολύπλοκων τεχνικών προβλημάτων, όπως το Shadow Mapping ή το Instancing, αλλά από εκείνες τις μικρές στιγμές που το σύστημα φάνηκε να αποκτά ζωή: όταν ένας χωρικός βρήκε μόνος του τον δρόμο ή όταν το φως του ήλιου γυάλισε πάνω στο ψηφιακό νερό.

Η ενασχόληση με τη συμβατότητα των συστημάτων (Windows/macOS) δίδαξε την αξία της προσαρμοστικότητας και της σωστής αρχιτεκτονικής. Το τελικό αποτέλεσμα είναι ένας "ζωντανός οργανισμός" κώδικα ~13.500 γραμμών, δομημένος με λογική και καθαρότητα. Όπως ο δημιουργός την έβδομη ημέρα "κατέπαυσε από πάντων των έργων αυτού", έτσι και τώρα, θέτω αυτόν τον ψηφιακό κόσμο υπό την ακαδημαϊκή σας κρίση. Ο κώδικας πλέον σιωπά, και το έργο παραδίδεται προς αξιολόγηση, με την ελπίδα πως η αρχιτεκτονική του θα αποδειχθεί στερεά και η λογική του άρτια, δικαιώνοντας τον κόπο κατασκευής τους.